RMIT UNIVERSITY

Assignment 2: Retrieval-Augmented LLM System for Automated Web Data Analysis Main Specification (Postgraduate)

Course	ISYS3476 (Postgraduate): Managing Semi-structured and Unstructured Data
Assessment	Assignment 2, individual
Weighting	40%
Due date	Wednesday, 21 October 2026 at 23:59 (Melbourne time).
Total marks	**40 points** (summary in Section 6)
Submission	Full-data CSV submissions to both Kaggle tracks, the Kaggle identity form, and one Canvas ZIP containing code, configs, evidence files, `WRITEUP.pdf`, and `DEMO.mp4`.
Late penalty	10% of the available marks per calendar day, up to five days. Work submitted more than five days late is not marked unless an approved extension, Equitable Learning Plan, or special consideration applies.
Environment	Python 3.10–3.14 inclusive, CPU-only marking, approved course-gateway models, and the packages listed in Section 2.5.

Read this first. The Main Specification defines assessed work and submission rules. The Appendix is the technical reference. The Rubric defines marking bands. Announcements and updated materials are available on Canvas.

Getting help. Use the discussion forum for questions that may help the cohort, workshops for implementation support, and email for personal matters. For setup and cross-folder imports, see the Canvas Python setup page and Python project imports page.

Learning goals and CLO alignment

This assessment addresses Course Learning Outcomes (CLOs) 1–5. It develops your ability to:

- **CLO 1:** apply information retrieval principles to locate relevant evidence using classical methods and modern LLM APIs.
- **CLO 2:** devise sparse, dense, and embedding-based indexing and retrieval pipelines.
- **CLO 3:** evaluate retrieval and analysis using reproducible experiments.

- **CLO 4:** design an LLM-assisted judging component for structured survey options.
- **CLO 5:** integrate indexing, retrieval, judging, and aggregation into a working pipeline.

Contents

1 Overview **3**
- 1.1 System goal . . . 3
- 1.2 Provided materials and data . . . 3
- 1.3 Baseline B0 . . . 3
- 1.4 Path to Pass: the minimum complete submission . . . 4

2 Task 1: System Improvement **4**
- 2.1 Required core changes . . . 4
- 2.2 Optional advanced methods for higher marks . . . 5
- 2.3 Required command interface . . . 5
- 2.4 Staff reproducibility check: mini run . . . 5
- 2.5 Environment, API use, and performance rules . . . 6

3 Task 2: Comparison Runs and Case Study **6**
- 3.1 Required comparison runs . . . 6
- 3.2 Case study . . . 7
- 3.3 LLM judge validation . . . 7

4 Task 3: Report, Evidence, and Demonstration **7**
- 4.1 Report requirements . . . 7
- 4.2 Runs Table . . . 7
- 4.3 Evidence trail . . . 8
- 4.4 Live demonstration . . . 8

5 Submission Platforms **8**
- 5.1 Kaggle: two tracks . . . 8
- 5.2 Canvas ZIP submission . . . 9

6 Assessment Summary **9**

7 Generative AI and Academic Integrity **9**

1 Overview

1.1 System goal

You receive a cleaned collection of public web posts, multiple-choice survey questions, precomputed full-collection document embeddings, and a runnable starter system named **B0**. Your goal is to improve B0 so it uses those posts as evidence to estimate public opinion over the exact answer options while staying within the course-gateway allowance of **USD $3 per student**.

In plain terms. For each survey question, your system should (i) retrieve and rank 100 supporting posts, (ii) select a subset for option judging, and (iii) estimate how opinions are distributed across the options. The distribution track compares your `distribution` with a hidden reference distribution. The evidence track checks whether relevant posts appear early in your ranked `supports` list using MAP@100.

Your system saves these results in a CSV file, with one row per question. Each row contains the question text and these two outputs:

- **`distribution`:** one non-negative probability per exact answer option, totalling 1.0 ± 10^{-6}.
- **`supports`:** exactly 100 unique document IDs, ordered by relevance for MAP@100.

1.2 Provided materials and data

Location	Materials
Canvas	Main Specification, Appendix, Rubric, report template, starter ZIP, full data package, API access guidance, Kaggle distribution and evidence tracks, and the Kaggle identity form.
Starter ZIP	Runnable B0, working and baseline configs, README, `requirements.txt`, mini data, and public self-check.

See the Appendix for data details and ethical use.

Full data. The full collection contains 183,303 documents. The question file contains 8 development questions and 10 hidden-test questions. Question text and exact answer options are visible for both splits. Reference distributions and relevant-document labels are not provided. Kaggle shows development scores and withholds hidden-test scores until the competitions close.

Mini data. The starter includes 1,000 documents, one question, and mini ground truth. Labelled supports are mixed with distractors for testing retrieval and judging. Mini scores guide debugging.

1.3 Baseline B0

B0 is a runnable index → retrieve → judge-and-aggregate pipeline that produces a valid CSV.

The working `mini_config.ini` and `full_config.ini` select retrieval and judging methods independently. Both initially select `b0`. The starter includes `bm25` and `dense` retrieval branches and an `llm_judge` judging branch for your implementations. Keep `mini_b0_config.ini` and `full_b0_config.ini` as unchanged B0 comparison configs.

In B0:

- the supplied B0 pipeline builds and searches a Whoosh index.
- boolean-style token matching returns deterministic supports in a fixed order.
- option judging uses token overlap.

- supplied aggregation logic converts document-level judgments into the final `distribution`.

Your job is to improve the weak parts: ranked retrieval for data access and LLM-based judging for data analysis. The Appendix gives a more detailed explanation of B0 and the supplied data.

Two key terms

- **Ranked supports:** exactly 100 unique documents returned by retrieval in relevance order.
- **Judged evidence:** unique documents selected from the ranked supports for option judging. B0 uses the first 20. You may choose a different subset or judge more documents. For LLM judging, more documents usually increase API use.

1.4 Path to Pass: the minimum complete submission

Path to Pass

Focus on these five steps first:

1. **Run B0** on the mini data to confirm that the starter and your environment work.
2. **Improve retrieval and ranking:** implement ranked retrieval and output relevance-ordered `supports`. Choose the starter's `bm25` or `dense` branch in `retrieval/rankers.py` (`retrieve_and_rank`) as your starting point.
3. **Improve evidence judging:** implement LLM-based judging. The starter provides an `llm_judge` branch in `judging/judges.py` (`judge_evidence`), and the supplied aggregation logic converts judgments into a valid `distribution`.
4. **Run Best and the three controlled comparisons on the full data using configs based on `full_config.ini`.** Submit these four runs' CSVs to both Kaggle tracks and record their development scores in the Runs Table (Section 4.2). The report template provides the B0 development scores; a full B0 run is optional.
5. **Check, report, and submit:** set `mini_config.ini` to the mini version of Best and complete both mini checks (Section 2.4). Complete the report, identity form, and Canvas ZIP.

2 Task 1: System Improvement

2.1 Required core changes

Best is assessed in data access and data analysis. It must include at least one meaningful method-level improvement in each area:

- **Data access:** improve B0 by replacing or extending its boolean retrieval with **ranked retrieval**. Return exactly 100 document IDs in `supports`, ordered by relevance. Either BM25 or dense cosine retrieval can meet this core requirement.
- **Data analysis:** improve B0 by replacing its token-matching judge with an **LLM** that assesses how the selected documents support the answer options. A single-pass structured prompt can meet this core requirement.

The starter provides code to build a Whoosh index for lexical retrieval. The supplied dense matrix contains document vectors rather than a completed dense search index, and the starter provides a helper for encoding comparable query text. Data analysis improvements must use an approved course-gateway LLM as the core judging component. Hyperparameter changes alone do not count as separate method-level improvements.

2.2 Optional advanced methods for higher marks

Advanced methods are optional unless you aim for the higher bands of Criterion C. Only the methods listed below count. For each claimed method, submit verifiable code and explain why you retained or rejected it. Other improvements, such as data preprocessing or parameter tuning, can improve performance but do not count as advanced methods.

- **Data access:** ranking fusion such as RRF, multistage retrieval or reranking, LLM-based query rewriting, MMR-based diversity, or ANN indexing.
- **Data analysis:** few-shot, chain-of-thought or step-back prompting; self-consistency; self-refine or verifier passes; automatic repair of invalid model replies; evidence-weighted judgment aggregation; or concurrent or asynchronous API calls.

2.3 Required command interface

Run these three commands from the extracted starter folder:

```
python -m index.build --config mini_config.ini --api_key YOUR_KEY
python -m rag_survey.retrieve --config mini_config.ini --api_key YOUR_KEY
python -m rag_survey.judge --config mini_config.ini --api_key YOUR_KEY
```

These commands use your working `mini_config.ini`, which selects retrieval and judging independently. Section 1.3 lists the supplied method names and unchanged B0 comparison configs.

1. `index.build` prepares the configured retrieval index or resources. B0 builds a Whoosh index. Mini and full document vectors are supplied for dense retrieval.
2. `rag_survey.retrieve` writes 100 ranked document records to `retrieved.jsonl`.
3. `rag_survey.judge` selects documents from the retrieved records, judges them, and aggregates the results. It saves `judgments.jsonl` and `submission.csv` in the configured `run_dir` and prints their full paths in the terminal.

Required interfaces. Keep the three command forms and final CSV schema unchanged. For full runs, use the same commands with `full_config.ini`.

Internal structure. You may add new modules or modify the supplied modules to suit your system. Week 10 provides examples of adding new modules, including modules that combine LLM-based query rewriting with RRF, pair few-shot prompting with a verifier pass, or apply evidence-weighted aggregation after judging. Keep the supplied INI format.

2.4 Staff reproducibility check: mini run

Before submission, run:

```
python -m test_sanity.check_submission --config mini_config.ini --api_key YOUR_KEY
```

Your submitted `mini_config.ini` must run the mini version of Best on the supplied mini data. The self-check runs indexing, retrieval and judging with this config. All three commands must finish within three minutes. If Best uses dense retrieval, use the supplied mini document embeddings.

Then check and evaluate the newly generated mini CSV:

```
python -m metrics.evaluate --submission runs/mini_best/submission.csv
```

This command uses the starter's default mini output path. If you changed the output location, use your actual mini CSV path. The evaluator checks the CSV against the supplied mini data

and prints the public distribution score and MAP@100. Both commands must finish without errors. Mini scores are for debugging. Full-data scores come from Kaggle.

Record the mini CSV path in the report template's Submission Check. Staff use these two commands with your submitted config and recorded output path, without modifying your code or config.

Start with mini

Before full runs. Confirm that the mini workflow runs reliably. Then move to full runs using `full_config.ini`.

Cost guide. For planning, allow about USD $0.01 for judging all 18 questions with `gpt-5-nano`, using one batch request with 20 full documents per question. At this estimate, USD $3 covers about 300 runs before retries and other API experiments. Actual cost depends on document and prompt length and the reply tokens.

Last year's usage. The median was 16 submissions on the distribution track and 12 on the evidence track. About 95% of students stayed at or below 40 submissions on each track.

2.5 Environment, API use, and performance rules

Environment. Your submission must run with **Python 3.10–3.14 inclusive** on CPU only. Use only the Python standard library and the supplied packages: NumPy, NLTK, the OpenAI Python client, Whoosh-Reloaded, BeautifulSoup, and FAISS-CPU. It must not require an external service other than the approved course gateway.

API use. Use only the approved models listed on the course API access page and remain within the **USD $3** allowance unless the course coordinator approves more. Use retrieval to select supports before LLM judging. Do not store API keys in source files or configuration files.

Performance rules. Your retrieval and judging methods must derive their results from the supplied questions and documents and work on unseen questions. Do not use hard-coded distributions, question-specific document IDs, fixed option positions, or lookup tables tailored to development or leaderboard cases. **These shortcuts receive zero** for the affected performance track, regardless of score.

3 Task 2: Comparison Runs and Case Study

3.1 Required comparison runs

After you have a Best run, complete these three controlled comparisons:

- **Access-Var:** disable, simplify, or swap one retrieval method or component, keeping judging fixed.
- **Analysis-Var:** disable, simplify, or swap one judging method or component, keeping retrieval fixed.
- **Config-Var:** choose either area, keep the same method, and change one configuration value, such as BM25's `k1` or `b`, ANN `nprobe` or `efSearch`, the LLM model, or the judged-document count.

For each comparison, copy Best's config, change the selected component or setting, and use a separate run folder. See the starter README for an example. Other advanced methods claimed for marking are assessed through their code and explanation.

Start each comparison from Best. If Best combines several improvements, change one component at a time and keep the others unchanged where possible. The alternative may be simpler or more complex than Best. Do not assume that a more complex method will perform better.

Comparison analysis. Use B0, Best and the three comparisons to justify your final choice. Connect each controlled change to specific retrieved documents or judgments and the resulting scores. For distribution, relate changes in predicted probabilities and answer-option ordering to the score, using the intuition in Appendix Section 4.1. For MAP@100, relate changes in relevant-document ranks to average precision. Discuss relevant trade-offs, such as retrieval quality versus runtime or judging accuracy versus API cost, and how an observed failure or limitation of a tested method informed your choice.

3.2 Case study

Analyse one development question by tracing its predicted `distribution` back to the retrieved evidence. Include the prediction and relevant top-ranked supports. Use two or three short excerpts, identified by document ID, to explain how the evidence supports or contradicts the predicted probabilities. Identify one limitation in the retrieved evidence or judging process that could have affected this prediction, and propose one concrete improvement.

3.3 LLM judge validation

Explain why you chose your LLM for judging. For full marks, support your choice with an accuracy comparison of two approved course-gateway LLMs using the same judging method. Use at least five manually labelled question-document examples from your retrieved development evidence. In this comparison, label each example with an exact answer option or `non_relevant`. Report both models' accuracy and explain how their errors, differences or agreement support your choice. Include the labels and model outputs in the Canvas ZIP.

4 Task 3: Report, Evidence, and Demonstration

4.1 Report requirements

Use the provided Word template, keeping its headings and structure, and submit it as `WRITEUP.pdf` in the Canvas ZIP.

Write no more than **3 pages**, excluding the cover page and Evidence Summary Table. Use 11pt for the main text and at least 9pt for tables.

Your report must include:

- a sufficiently complete description of Best and any advanced methods claimed for marking, including why each was retained or rejected.
- the Runs Table, comparison analysis and case study required in Task 2.
- the justification for your LLM judge choice (Section 3.3).
- an Evidence Summary Table (Section 4.3).

4.2 Runs Table

Include at least these five Runs Table rows. Submit Best, Access-Var, Analysis-Var, and Config-Var to both Kaggle tracks. The report template provides the B0 development scores; a full B0 run and Kaggle evidence are optional.

Row	Meaning
B0	Provided baseline.
Best	The final system described and analysed in your report.
Access-Var	Best with the data-access method comparison (§3.1).
Analysis-Var	Best with the data-analysis method comparison (§3.1).
Config-Var	Best with one configuration comparison in either area (§3.1).

Each row must include the data-access and data-analysis settings or changes, development distribution score, development MAP@100, and approximate API token count, cost in USD, or both. Use 0 for runs with no API calls. You may include additional informative runs. API use may vary between repeated runs.

To support runtime or memory claims, add the relevant **Runtime (s)** or **Peak memory (MB)** columns to the Runs Table, using measurements from the same workload and conditions.

4.3 Evidence trail

The Runs Table records comparisons and scores. The Evidence Summary Table locates method implementations and supporting run files in the Canvas ZIP.

For each Kaggle run cited in the report, keep its JSONL, CSV, and cropped Kaggle screenshots together in its folder under `runs/`. Name the screenshots `kaggle_distribution.png` and `kaggle_evidence.png`. Identify the folder in the table rather than embedding screenshots in the report.

In the Evidence Summary Table, give the ZIP-relative code path and function or class name for each method claim, plus the config and run-folder paths for results.

Evidence trail
Report Quality receives zero if the Evidence Summary Table is missing. An incomplete table, or one that makes the supporting files difficult to locate, cannot receive full marks for Report Quality.
Reported methods and results must match the submitted code, configs and saved outputs. Fresh LLM calls may vary. Unsupported results are excluded from marking. Substantial contradictions may result in zero for the affected reporting component.

4.4 Live demonstration

Submit a screen recording named `DEMO.mp4`. **Maximum length: three minutes.** Use one continuous, unedited take with live narration. Edited recordings receive zero for this component. Accent and presentation polish are not assessed. Brief pauses and live corrections do not prevent full marks.

Open the submitted project and run either the mini or full system. Show one result on screen, then open a relevant submitted code block and explain how the retrieval or judging mechanism contributed to that result. For the highest band, trace one question through retrieval, judging, and the final CSV, explaining how the intermediate outputs produced the displayed result.

5 Submission Platforms

5.1 Kaggle: two tracks

Upload each required run's CSV to both Kaggle tracks:

- the **distribution track** evaluates `distribution`.
- the **evidence track** evaluates `supports` using MAP@100.

Kaggle identity form. Use the same non-identifying Team Name on both tracks and submit it through the Kaggle identity form by your applicable deadline. Do not use your real name, student ID, or RMIT email as the Team Name. The Kaggle username or handle is optional. An unmatched Team Name cannot receive Kaggle marks.

Deadlines and competition closing date. Your deadline is 21 October 2026, or your approved extension deadline. Both Kaggle competitions remain open until 23:59 on 4 November 2026 (Melbourne time) for approved extensions. This later closing date does not extend your deadline. Hidden-test leaderboards are revealed when the competitions close.

Final submissions. Select up to two final submissions per track. If you select fewer than two, Kaggle automatically adds your other valid submissions with the highest development scores, up to two in total. If you have only one valid submission, it uses that one. Your private-leaderboard score is the highest hidden-test score among these submissions. Final selections may differ between tracks.

Submission time. Late penalties are calculated using the latest upload time among your Canvas ZIP and the Kaggle submissions used for marking on both tracks.

Submission snapshot. We capture the leaderboard at your applicable deadline and use this snapshot for marking. For late submission within the five-day late window, we use the corresponding Kaggle results at submission time and apply the late penalty.

5.2 Canvas ZIP submission

Submit the authoritative code and report ZIP as `YourName_s1234567_ass2.zip`.

After extraction, the ZIP must open at one clear starter-folder root and include:

- source code, `requirements.txt`, `mini_config.ini`, and every config cited in the report.
- `WRITEUP.pdf` and `DEMO.mp4`.
- compact evidence files needed to verify the Runs Table, case study, and LLM judge validation.
- the supplied mini data and public self-check.

Before creating the ZIP, clear temporary API responses and generated embeddings from `cache/`. Keep report evidence and run outputs under `runs/`, and the supplied mini embeddings under `data/mini/`. Exclude `.venv/`, the full corpus, full document embeddings, generated full indexes, and unrelated files. Files outside the submitted ZIP are not marked.

6 Assessment Summary

ID	Assessed component	Pts
A	Kaggle performance (development and hidden-test distribution and evidence scores)	12
B	Mini reproducibility check and live demonstration	6
C	Method design and description (data access + data analysis)	8
D	Controlled experiments, case study, and LLM judge validation	8
E	Report quality	3
F	Code quality	3
	Total	**40**

Before submitting. See the Rubric PDF for full bands. Complete the report template's Submission Check.

7 Generative AI and Academic Integrity

Condition 4 applies to approved course-gateway model calls within your submitted system.

Condition 2 applies to the submitted code and report. Generative AI may help with grammar, spelling, clarity, and formatting. It must not generate or rewrite code, algorithms, experiments, analysis, or report content.

On the report cover, declare each model and its purpose, including the required LLM judge and any permitted language-polish use. Keep original prompts and pre-polish writing for possible moderation. Provide full chat records only if requested. See the Canvas Generative AI Use in Assessment page. This specification sets the assignment conditions.

Standard RMIT academic integrity requirements apply. Do not share code, implementation-revealing prompts, report text, Kaggle submissions, or private assessment data.